

凌晨两点的告警

一款只有两千行代码的框架，如何让软件学会“反悔”

本文事实与代码来源：cordiverse/cordis 源码 (commit 8cc9e33) · 《时空可组合性编程范式》论文 (Shi, Zhang, Cui, PKU × DeepSeek, 2026, 88 页) · 《可逆的插件系统》(Sigma, 2023) · Koishi / DeepSeek Harness 官方文档

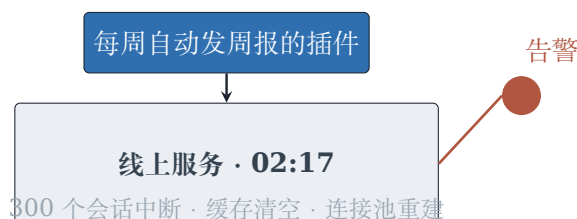
00 · 事故

凌晨两点十七分，值班群里弹出一条告警。

线上聊天机器人服务刚刚执行了一次插件热更新——那个每周自动发周报的插件被卸载了，版本升级，本是一次再平常不过的运维操作。但三分钟后，监控大屏上一片飘红：正在进行的 300 个用户会话全部中断，缓存清空，连接池重建，服务端了整整四分钟才恢复。

复盘报告只有一行结论：

现代框架没有为“拆除”设计一条路。卸载一个插件，必须重启整个进程。



卸载一个插件，为什么必须重启整个进程？

这句话听上去荒诞——我们生活在一个能造出大模型的时代，软件却连“把装上去的东西拆下来”都做不利索。更荒诞的是，这不是个例。Visual Studio Code 的用户一定深有体会：按装机量排前一百的扩展里，**87 个含可执行代码的扩展，卸载时都必须重启整个编辑器**（2026 年 6 月 Marketplace 数据）。Koa 注册了中间件就没有 API 能撤销，Vue 注册了组件就没有 API 能卸载，Node.js 加载了 .node 原生模块，这个文件就被进程永久占用，直到它死去。

我们的问题随之而来：**软件为什么只会“装”，不会“拆”**？更进一步：如果有一天软件要开始修改自己——AI 自己生成组件、替换组件——“拆不下来”就不再是运维的烦恼，而是一票否决的死刑。

回答这个问题的人不多，但其中一伙人的答案已经跑在了生产环境里：一个叫 Sigma 的开发者，从 2017 年写聊天机器人框架开始，花了八年时间，把这个答案打磨成一个只有 **1848 行 TypeScript** 的插件框架，名叫 **Cordis**。2023 年他把它写成一篇设计文章，2026 年他和北大、DeepSeek 的合作者把它写成了 88 页的可证明理论；同年 8 月，DeepSeek 用它做了自己智能体框架 dsh 的内核——发布当天 **42k stars**。

这场事故的开头和结尾之间，隔着三个问题的拆解和七次思路的迭代。

01 · 拆不掉的软件，病在三处

先把“拆不掉”拆开看。任何一个运行中的组件，都在做三件事：**对环境动手**（打开文件、注册监听、占用端口），**向环境伸手**（依赖别人提供的服务），**与环境共存**（多个组件交织在一起）。这三件事，对应三处病灶。

第一病，时间维度：动手之后，没有人收拾。

组件运行时做的每一件事，都是对共享环境的修改——专业术语叫副作用（Side Effect）。修改是永久的，除非有人明确地改回去。而现代框架的系统性失职在于：**它给了你注册的 API，却没有给你注销的 API。**



共同病灶：缺少一个“可逆”的抽象

这一病还会传染。B 插件监听 A 插件注册的事件；你卸载了 A，B 还竖着耳朵——之后它收到的每一条消息都来自一个不存在的源头。这种“卸载残留”，是生产环境里最难查的一类 bug。

第二病，空间维度：伸手之前，没人排队。

组件与组件之间有依赖：B 要用 A 提供的数据库。现实中依赖管理是一片丛林——A 还没启动 B 就来了，B 空手而归当场崩溃；A 中途被卸载，B 毫无察觉，下一次调用撞上悬崖；A 被悄悄换成 A'，B 还攥着旧引用，一问三不知。

传统的土办法叫“加载顺序”：先 A 后 B，把空间问题硬编码进时间顺序。这法子在小系统里能凑合，但 Koishi 生态有 4000 多个插件——4000 个组件手排依赖顺序，相当于让调度员徒手排 4000 班火车的进站时刻。

第三病，组合维度：折腾的过程，留下了指纹。

最隐蔽的病是“路径依赖”：同一套插件，加载顺序不同，最终行为就不同。你在本地复现不了生产的 bug，因为两边加载顺序差了那么一点。理想的状态是**路径无关**——不管中间怎么折腾（装了又卸、卸了又装、换了顺序装），最终行为只取决于最终启用了哪些插件，过程不留指纹。

这三处病灶的病灶，其实只有一个：**软件缺少一个“可逆”的抽象**。剩下的事情，都是顺着这个洞见往下走。

02 · 撤销按钮、账本与叠盘子

第一个洞见朴素得近乎平庸：**把“怎么拆”和“怎么装”写在一起。**

```
// 传统写法：开了端口，没人知道怎么关
function serve(port: number) {
  const server = createServer().listen(port)
}

// 可逆写法：怎么关，跟着怎么开走
function serve(port: number) {
  const server = createServer().listen(port)
  return () => server.close() // <- 撤销按钮
}

const dispose = serve(80) // 启动，顺手拿到撤销按钮
dispose() // 按下，一切还原
```

规则只有一条：每个原子操作，正向一步，反向一步，**反向必须和正向写在一起**。这个原则叫局部性——拆装逻辑放同一处，开发者才忘不掉。VSCode 的 deactivate 钩子之所以没用，恰恰死在局部性上：创建的代码在 activate 里，销毁的代码在几千行之外的另一处，中间漏写一行，就是一次静默泄漏。

但“靠自觉”从来不是工程的答案。Cordis 的做法是：**把记账变成框架的职责**。所有会改变环境的操作必须经过唯一的入口 `ctx.effect`，框架自动收集你产生的每一个撤销按钮，卸载时**逆序**执行：

```
// Cordis 真实源码 (fiber.ts, 节选)
const dispose = () => {
  for (const dispose of disposables.splice(0).reverse()) {
    // <- 这行 .reverse() 就是全部精髓
    task = task ? task.then(dispose) : dispose()
  }
  return task
}
```

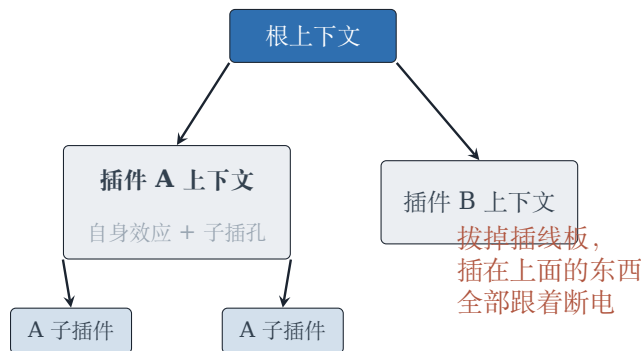


为什么要逆序？想想叠盘子：先装插件 A（打开文件），再装插件 B（往文件里写配置）。撤销时必须先撤 B 的写入、再关 A 的文件——顺序一反，B 就会往已经关闭的文件里写东西。后进先出，就这么简单。一行 `.reverse()`，就是整个可逆性理论的工程落点。

03 · 插线板、守卫与惯性

账本解决了“收拾”，但还有两个问题悬着：副作用**属于谁**，依赖**谁来管**。

属于谁的答案，藏在 Cordis 作者的一个比喻里：上下文是插座，插件是插头。墙上的插座是根上下文；插线板是插在插座上的“插座”——它自己用电，又提供新的插孔；插线板上还能再插插线板。**拔掉一个插线板，插在它上面的所有东西跟着断电。**



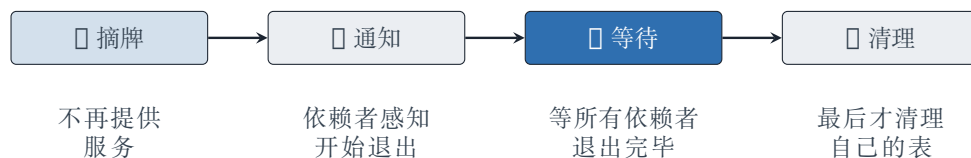
翻译成代码：每加载一个插件，就从当前上下文派生一个子上下文（实现上就是 JavaScript 的原型链继承，五行代码）。子插件的一切副作用记在子上下文上，插件整体作为父上下文的一个副作用。于是“卸载父插件级联卸载子插件”不是需要专门实现的特性，而是**数据结构自带的属性**。一个给别的插件提供 API 的插件，占用的资源是“能占用资源的资源”——数学上这意味着上下文类型必须是递归的，工程上它只是这五行原型链代码。

谁来管依赖的答案更直接：依赖关系不靠顺序，靠**声明**。插件写 `inject: ['database']`，框架保证 `database` 就绪前它不会启动，`database` 消失时它会先退出。提供方换实现、甚至运行时被替换，依赖方自动感知、自动重激活——Koishi 那 4000 个插件之所以能协作，就是因为作者之间不需要任何沟通，只需要对“`database` 这个名字”达成共识。

最精彩的是**卸载一个还在被依赖的服务**——这是全系统最容易死锁的角落，Cordis 只用四步解决（真实源码，`reflect.ts`）：

```
return async () => { // 卸载时执行：
  delete this.store[key] // (1) 先摘牌： "我不再提供这个服务了"
```

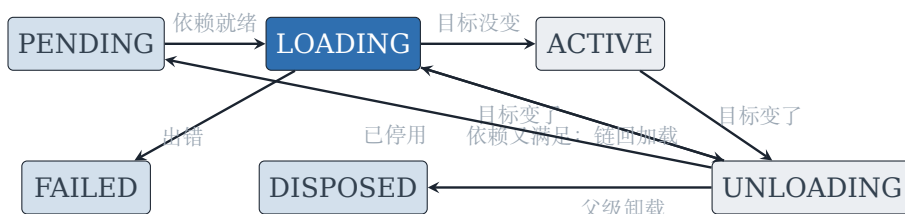
```
const fibers = this.notify([name]) // (2) 依赖者们感知变化，开始自己的退出流程
await Promise.allSettled( // (3) 守卫：等所有依赖者退出完毕
  fibers.map(fiber => fiber.await())
)
delete this.ctx.fiber.store![name] // (4) 全部退出后，才清理自己的表
}
```



绑定在等待期间保持可用

这四步的顺序是精心设计的：先摘牌，依赖者立刻感知“服务没了”并开始退出；但值还在表里，**退出途中仍可访问**——连接池还在，正在退出的 B 还能把连接还回去。等全部落定，才清理自己的表。死锁不会发生，因为摘牌之后依赖者必然退出，等待者等的只是“已经不可满足”的纤维。

最后一块拼图是**惯性**。加载和卸载可能耗时（异步 I/O），迁移进行到一半，依赖关系又变了怎么办？Cordis 的答案是每个插件实例跑一个六态状态机——PENDING（等依赖）→ LOADING（执行插件）→ ACTIVE（运行）→ UNLOADING（执行撤销）→ 回到 PENDING 或 DISPOSED，另加一个 FAILED 兜底。两条铁律：迁移一旦启动就**跑到底**（飞行中的异步操作不可安全中断，中断了就是半加载状态）；加载完成时目标又变了，就**原地链入卸载**，卸载完成时依赖又满足了，就**原地链回加载**。



惯性：迁移一旦启动就跑到底

这七条思路——撤销按钮、自动记账、LIFO、上下文树、依赖声明、守卫、惯性状态机——合起来就是 Cordis 的全部。1848 行，没有魔法。

04 · 为什么需要 88 页数学

一个自然的疑问：一个 1848 行的工程框架，为什么还要配一篇 88 页的论文？数学是不是装饰品？

不是。每一块数学，都对应一个真实的工程 bug：

- **幺半群**（封闭性 + 结合律）：保证插件组合无歧义。没有它，两个插件互调顺序行为不同，系统不可预测。
- **群**（每步有逆）：保证每步可撤销。没有它，就是 VSCode 那 87% 的现状。
- **同态** ($\text{effect}(f \circ g) = \text{effect } f \circ \text{effect } g$)：保证分开记账等于合并记账。没有它，复合插件的撤销链会错，热更新后状态错乱。
- **汇合性**：保证路径无关——最终状态只取决于最终配置。没有它，凌晨两点的生产 bug 就永远无法复现。

论文还证明了两件听起来像操作系统教材的事：**无死锁**（守卫一定会释放）和**收敛唯一**（无论怎么装卸，都收敛到同一个静止状态）。这些命题第一次在插件系统上被完整证明——这就是那 88 页的价值：**把“我猜它不会出错”换成“我证明它不会出错”**。

当然，Cordis 也是诚实的。它划了一条边界：打开文件、建立连接，这些可以撤销（边界内）；发出去的 HTTP 请求、写进外部系统的数据，这些无法撤销（边界外）——你没法撤回一封已经发出的信。边界外只有两个办法：扣留（等状态确定再发）或补偿（发错了发一封更正）。这是任何“可恢复系统”都绕不开的设计判断。

05 · 当软件开始修改自己

让我们回到凌晨两点事故现场。

在那场事故里，动手的还是一名人类运维。但 2026 年发生了一件更有趣的事：8 月 13 日，DeepSeek 开源了 Harness (dsh) ——一个智能体框架，它的全部 54 个包——agent loop、工具注册表、模型适配器——**都是 Cordis 插件**。含义是：会话进行到一半，可以热替换模型适配器而不丢上下文。

论文的结论部分点明了这条线的终点：**自进化智能体**。想象一个会自己写代码的 AI：它今晚决定优化自己的一个工具函数，于是加载新代码、替换旧组件。如果每次自我修改都要重启，它会反复打断自己的任务、丢掉所有上下文；如果它某次改坏了——比如把恢复程序本身改挂——重启大法连自救都做不到。

所以那个 2017 年开始写 QQ 机器人的年轻人是对的：可逆性从来不只是运维的方便。当软件开始修改自己，“拆得下来”就从锦上添花的框架特性，变成了**进化能力本身的前提**。Cordis 的三个性质恰好是解药：可逆（改坏了自动回滚）、响应式（依赖变化自动重排）、汇合（无论怎么折腾，最终状态只取决于最终配置）。

把时钟拨回凌晨两点十七分。如果那套服务跑在 Cordis 上，卸载周报插件会是这样的：旧插件进入 UNLOADING，撤销它注册的一切，300 个会话原封不动，缓存一个字节不少，整个过程不出一秒。值班群安安静静，那场事故根本不会发生。

软件花了五十年学会“装”，而它刚刚开始学“拆”。当一个系统既能装也能拆，它才第一次拥有了修改自己的资格——这是所有关于“AI 自进化”的宏大叙事里，最不起眼、也最不可或缺的一块地基。

本文为叙事版技术文章 · 技术细节与完整代码映射见《让软件可以“安全地反悔”：可逆插件系统科普》
源码：github.com/cordiverse/cordis · 论文：github.com/cordiverse/paper（中文全译本随附）